

Problem 2: Determine the Order of Growth (20 pts)

Answer the following questions in a separate README, PDF, or Word document.

For each code fragment below, state the **time complexity using Big-O notation** and briefly explain why.

1. Matrix Traversal

```
for (int i = 0; i < r; i++) {  
    for (int j = 0; j < c; j++) {  
        cout << A[i][j] << " ";  
    }  
}
```

Time Complexity: $O(r \times c)$

Explanation:

The outer loop runs `r` times and the inner loop runs `c` times for each row.

Total operations = `r × c`.

2. Matrix Addition

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        C[i][j] = A[i][j] + B[i][j];  
    }  
}
```

Time Complexity: $O(n^2)$

Explanation:

There are two nested loops, each running `n` times.

Each matrix element is processed once.

Total operations = `n × n = n<sup>2</sup>`.

3. Matrix Multiplication

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        int sum = 0;  
        for (int k = 0; k < n; k++) {  
            sum += A[i][k] * B[k][j];  
        }  
        C[i][j] = sum;  
    }  
}
```

Time Complexity: $O(n^3)$

Explanation:

There are three nested loops, each running n times.

Total operations = $n \times n \times n = n^3$.

This is the standard complexity of classical matrix multiplication.

4. Doubling Loop

```
int i = 1;
while (i < n) {
    i *= 2;
}
```

Time Complexity: $O(\log n)$

Explanation:

The value of i doubles each iteration:

$1, 2, 4, 8, 16, \dots$

The number of times you can double before reaching n is approximately $\log_2 n$.

5. Subset Generation Using Bitmasking

```
for (int mask = 0; mask < (1 << n); mask++) {
    for (int j = 0; j < n; j++) {
        if (mask & (1 << j)) {
            cout << arr[j] << " ";
        }
    }
}
```

Time Complexity: $O(n \times 2^n)$

Explanation:

- * The outer loop runs for all subsets of a set of size n .
- * Number of subsets = 2^n .
- * For each subset, the inner loop checks all n elements.

Total operations = $n \times 2^n$.

This is exponential complexity.